

```
"rgba": [0,255,0,1],
"hex": "#0F0"
}
}}
```

You can load the same in R using the *fromJSON()* function, as shown below:

```
> colors <- fromJSON(txt="colors.json")
> colors
  color category    type      code.rgba code.hex
1 black      hue primary 255, 255, 255, 1  #000
2 white     value  <NA>      0, 0, 0, 1  #FFF
3 red       hue primary  255, 0, 0, 1  #FF0
4 blue      hue primary   0, 0, 255, 1  #00F
5 yellow    hue primary  255, 255, 0, 1  #FF0
6 green     hue secondary 0, 255, 0, 1  #0F0
```

The ‘fromJSON’ function has support for a number of arguments:

- *txt*: This is a JSON string, URL or a file name that contains JSON.
- *flatten*: This is a Boolean with TRUE or FALSE that automatically flattens nested data frames.
- *dataframe*: This specifies how to encode *data.frame* objects. It should be either ‘rows’, ‘columns’ or ‘values’.
- *matrix*: Encoding and higher dimensional arrays should either be ‘rowmajor’ or ‘columnmajor’.
- *Date*: Formats that are applicable are ‘ISO8601’ or ‘epoch’.
- *null*: Values within a list must either be ‘null’ or ‘list’.
- *na*: Values can be ‘null’ or ‘string’.
- *digits*: Specifies the maximum number of digits after the decimal point that should be printed.
- *pretty*: Indentation with whitespace for JSON output.

You can get the type of the ‘colors’ object using the *typeof()* function as shown below:

```
> typeof(colors)
[1] "list"
```

The number of rows, columns and column names are shown in the following output:

```
> colnames(colors)
[1] "color" "category" "type" "code"
> nrow(colors)
[1] 6
> ncol(colors)
[1] 4
```

You can obtain the values in the ‘color’ column as follows:

```
> colors$color
[1] "black" "white" "red"   "blue"  "yellow" "green"
```

The hex values can be obtained using nested invocation, as shown below:

```
> colors$code$hex
[1] "#000" "#FFF" "#FF0" "#00F" "#FF0" "#0F0"
```

Database

There are two major interfaces, the R Open DataBase Connectivity (RODBC) and R DataBase Interface (DBI), available in R to interact with databases. The ‘RSQLite’ package will be installed to demonstrate connectivity using the DBI interface to an SQLite database.

```
> install.packages("RSQLite")
```

You now need to load the installed library using the following command:

```
> library("RSQLite")
```

A driver object needs to be initialised with the *dbDriver()* function to indicate the actual driver to use when connecting to the database.

```
> driver <- dbDriver("SQLite")
```

You can then create the connection to the database using the *dbConnect()* function as shown below:

```
> connection <- dbConnect(drv=driver, dbname="Chinook_Sqlite.
sqlite")
```

The DBI specification recommends the use of the following arguments for authentication.

- *user*: This is a valid user name in the database, and the default value is the current user.
- *password*: This is the login password to be supplied.
- *host*: This is the name of the machine to connect. The default value is the local connection.
- *port*: Refers to the port number. The default is the local connection.
- *dbname*: Is the name of the database on the host or the database file name.

The details about the connection can be obtained using the *dbGetInfo()* function, which takes in the connection object as an argument.

```
> dbGetInfo(connection)
$db.version
```